```python
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

-
- Jewelry Planner (`/generate-jewelry`): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```python
# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result
@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
```

```
    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

  - User Auth (`/register`, `/login`, `/logout`, `/token`): Manages user registration, authentication, and session control.
  - Session Info (`/session-info`, `/session-data`): Retrieves user-specific session data for personalized AI interaction.
- **Activity 3.2: Modular Architecture Setup**

`/recommendations-details`: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```python
# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")
```

- **Activity 3.3: CORS & Static Routing**

`/history`: Retrieves the user's past recommendation queries and results for review or re-use.

```python
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

- **Activity 3.4: Startup and main function** `/startup`: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```python
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800  # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when the script is run directly.

```python
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

MILESTONE 4: UI Development

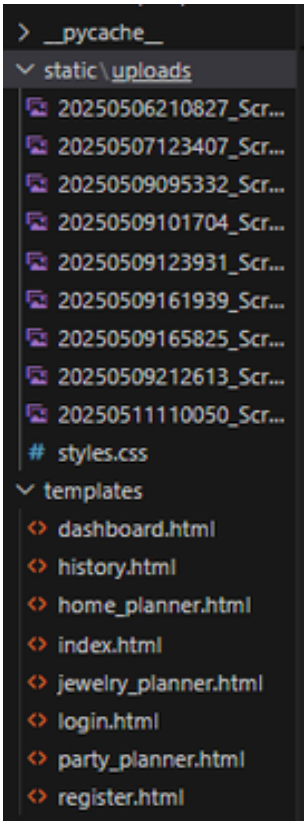
This milestone focuses on developing a **lightweight, responsive, and intuitive frontend** for **PocketSmart AI** using **HTML**, **CSS**, and **Jinja2 templates**. The interface

enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering.The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



MILESTONE 5: Testing & Optimization